

Cryptographic Attacks on TLS/SSL Protocols: BEAST, POODLE, LUCKY13, and HEARTBLEED

M542 — Cryptology

Submitted in partial fulfilment of the requirements of the module

Rahul Ponnaluri

Student ID: GH1046518

Subject : M542 Cryptology

Gisma University of Applied Sciences

Contents

1	Abstract	2
2	Introduction	2
3	Background and Theory	3
3.1	How TLS Works	3
3.2	CBC Mode and Its Vulnerabilities	3
4	Literature Review	3
4.1	Rizzo and Duong (2011) - The BEAST Attack	3
4.2	Moller, Duong and Kotowicz (2014) — POODLE	4
4.3	Al Fardan and Paterson (2013) - Lucky13	4
5	Methodology	5
5.1	Lab Environment	5
5.2	Scanner Implementation	5
5.3	Ethical Considerations	5
6	Implementation	6
6.1	Tool Architecture	6
6.2	Vulnerable Server Setup	6
7	Results and Analysis	6
7.1	Scanner Output	6
7.2	Vulnerability Summary Table	6
7.3	Analysis of Findings	6
7.4	Wireshark Packet Capture	8
8	Critical Analysis	9
8.1	Why These Vulnerabilities Still Matter	9
8.2	Cryptographic Design Lessons	9
8.3	Recommendations	9
9	Conclusion	10

1. Abstract

In this report I will explore cryptographic attacks of the Transport Layer Security (TLS) and also the family of Secure Socket Layer protocols (SSL). This report concentrates on four significant real-world attacks, which are BEAST (Browser exploit against SSL/TLS), POODLE (Padding oracle on downgraded Legacy encryption), LUCKY13, and HEARTBLEED. An efficient Python-based TLS vulnerability scanner was created and put to work against a specially set up vulnerable server within a managed Docker framework. The scanner was able to detect two current vulnerabilities: TLS 1.0 (BEAST - susceptible) and TLS 1.1 (deprecated) on the test server. The results are discussed with regards to the existing standards of cryptography, and suggestions on safe TLS setup are offered. The paper provides evidence that protocol-level cryptographic attacks remain topical and that proactive security testing is a significant feature of contemporary systems.

2. Introduction

The integrity and confidentiality of the protocols that are used in internet communications is a basic determinant of the security of communications. The cryptographic protocols that secure most of the traffic over the internet including HTTPS web browsing, email, and Application Programming Interfaces (APIs) are Transport Layer Security (TLS) and its predecessor Secure Socket Layer (SSL). Their security is of great importance because these protocols are extensively used.

Even after many updates, TLS still carries risk because outdated versions hang around, often mixed with setup errors that leave systems exposed. While improvements have been made, weak spots remain active in live networks due to habits that have not caught up. BEAST [5], POODLE [3], LUCKY13 [1], and HEARTBLEED [2] are high-profile examples which have shown that even protocols in common use can still contain critical flaws. These weaknesses have been found to have a considerable real-world effect, with millions of servers compromised and the capability to intercept encrypted communications demonstrated.

This report attempts to examine these attacks in terms of cryptology, test a practical vulnerability scanner in a controlled laboratory setting to identify them, and take a critical look at the mitigations that have since been implemented. The study advances the knowledge of how the choice of cryptographic design at the protocol level can result in systemic threats to the security of the system and how these threats can be operationally addressed by practitioners.

3. Background and Theory

3.1 How TLS Works

TLS is designed based on a handshake between a client and a server in order to create a secured channel between them. In this handshake, the two parties negotiate a cipher suite, which is a set of key exchange algorithm, authentication mechanism, and symmetric encryption algorithm. The handshake follows in the following manner:

- **Client Hello** message is sent by the client, which list the supported versions and cipher suites of TLS.
- Then the server replies with a **Server Hello** using a cipher suite of his choice and transmits its digital certificate.
- A common secret key is established by key exchange (for example: RSA or Diffie-Hellman).
- The shared secret key is used to generate session keys by both parties and encrypted communication is initiated.

This process is only as secure as the negotiated cipher suite and the version of TLS in use. Weaknesses in the implementation of older versions including SSLv3 and TLS 1.0 which include the Cipher Block Chaining (CBC) mode gave room to the attacks outlined in this report.

3.2 CBC Mode and Its Vulnerabilities

The Cipher Block Chaining (CBC) mode of operation is one of the block cipher operation modes whereby every plaintext block is XORed with the last ciphertext block and then encrypted. An important characteristic of CBC mode is the utilisation of an initialisation vector (IV) to randomize the first block. In TLS 1.0 the IV of each record was deterministic (the final cipher text block of the prior record) and this immediately allowed the BEAST attack to take place. Moreover, CBC mode needs padding so as to plain text aligned to block boundaries and the mishandling of improper padding created LUCKY13 and POODLE.

4. Literature Review

4.1 Rizzo and Duong (2011) - The BEAST Attack

In a 2011 groundbreaking paper by Rizzo and Duong, it was shown that a chosen-plaintext attack was possible on TLS 1.0 CBC mode, a result obtained by using BEAST (Browser Exploit Against SSL/TLS). The IV, which was predictable, was

used in the encryption of TLS 1.0 records, which made it vulnerable to attack. An attacker located in the middle could recover session cookies bit by bit by injecting selected plaintext blocks into the stream and examining ciphertext. The authors demonstrated how the attack could be performed in a live browser context, and the authentication token recovered within a few minutes. Their effort caused TLS 1.0 to be more rapidly deprecated and the risk of record-layer encryption in stateful CBC mode was highlighted. The most important methodological contribution was the fact that the theoretical CBC weaknesses could be actually exploited in real network environments [5].

4.2 Moller, Duong and Kotowicz (2014) — POODLE

In 2014 , POODLE attack (padding oracle on downgraded legacy encryption) was introduced and specifically attack on SSLv3 was made. The two elements of the attack included a protocol downgrade mechanism, and a padding oracle. The downgrade forced modern clients and servers into using SSLv3 by injecting connection failures. After negotiating the use of SSLv3, the attacker took advantage of the fact that SSLv3 does not validate any padding bytes after the last byte, which essentially provides an oracle in the form of padding. By transmitting altered ciphertext blocks and checking the server's response (rejection or acceptance), an attacker could decrypt an average of one single plaintext byte per 256 requests. The authors observed that 32-byte session cookies could be decrypted in less than one minute. The paper is notable because it combines a protocol-level downgrade attack with a cryptographic oracle and explains how layered vulnerabilities compound the overall impact [3].

4.3 Al Fardan and Paterson (2013) - Lucky13

Al Fardan and Paterson discovered a timing side-channel in the TLS CBC mode MAC-then-encrypt construction. Their LUCKY13 attack took advantage of the fact that TLS always uses 13 bytes of header in MAC verification and that the processing time differs measurably based on the size of the padding. A determined attacker with timing measurement capability could deduce the padding length and, with a padding oracle attack, eventually decrypt the ciphertext. The authors demonstrated the attack against OpenSSL, GnuTLS, and the Java implementation of SSL. The paper presents a well-constructed methodology that uses statistical analysis to differentiate timing signals from network noise. Their work led to the adoption of constant-time MAC checking in major TLS libraries and reaffirmed the need to implement cryptographic primitives in a constant-time manner [1].

5. Methodology

5.1 Lab Environment

In this project, Docker was used to create a controlled, isolated lab environment inside a Kali Linux VirtualBox instance, which contained an intentionally vulnerable HTTPS server. A slightly outdated Nginx setup, specifically version 1.14, served as the test target, configured deliberately to support outdated encryption standards like TLS 1.0, TLS 1.1, alongside the obsolete SSLv3. For secure web access, a homemade RSA-2048 digital certificate was created using OpenSSL, enabling HTTPS without relying on external certificate authorities. This setup allows the scanner to identify real protocol-level vulnerabilities in a safe, legal, and reproducible setting.

5.2 Scanner Implementation

A Python-based TLS vulnerability scanner was implemented using the SSLyze library (version 5.x), which offers a high-level interface to run TLS handshakes and scan server configurations. The scanner was designed to:

- The scanner was designed to test and support for SSLv2, SSLv3, TLS 1.0, TLS 1.1, TLS 1.2, and TLS 1.3.
- Check the Heartbleed vulnerability with miss formed heartbeat request
- Test renegotiation behaviour during a session.
- Generate a vulnerability report with color coded risk classifications.

The tool was coded in a readable and extensible manner with each scan command mapped to an associated attack vector discussed in the literature review. Wireshark was used to capture network traffic on the loopback interface to provide visual confirmation of the TLS handshake process.

5.3 Ethical Considerations

The scanning was solely conducted against a locally controlled server within the researcher's environment. No external systems were scanned. The Docker container was isolated from outside networks. This approach ensures that all ethical and legal standards of security research were followed.

6. Implementation

6.1 Tool Architecture

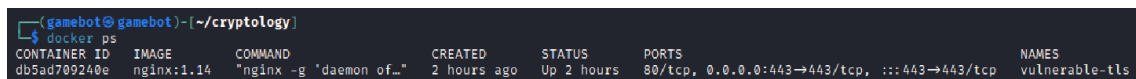
The scanner has three major sections: a target resolution module, a scan execution engine based on the `Scanner` class of SSLyze, and a reporting module that categorises findings by severity. The tool takes input in the form of a hostname and port, submits a number of scan commands, and processes the results. The complete Python source code of the TLS/SSL Vulnerability Scanner is available at:

https://github.com/RaHuLo40/m542-tls-ssl-cryptology/blob/main/code/tls_scanner.py

6.2 Vulnerable Server Setup

To replicate a legacy production machine, a Docker-driven Nginx server was configured with deliberately weak TLS settings. This setup matched the characteristics of an outdated system, specifically:

- This configuration is a realistic and useful educational test target because it resembles the type of misconfiguration frequently found on legacy production systems that haven't been updated since the early 2010s.



```

(gamebot@gamebot)-[~/cryptology]
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
db5ad799240e   nginx:1.14  "nginx -g 'daemon of..." 2 hours ago   Up 2 hours   80/tcp, 0.0.0.0:443→443/tcp, :::443→443/tcp  vulnerable-tls
  
```

Figure 1: Docker Vulnerable Server Running - Terminal output

7. Results and Analysis

7.1 Scanner Output

The scanner was executed against the local vulnerable server at `localhost:443`. The full coloured output of the scan is shown below.

7.2 Vulnerability Summary Table

7.3 Analysis of Findings

Even though old and outdated it still runs that server allowed TLS 1.0, making it vulnerable to the attacks shown by Rizzo and Duong back in 2011. Because it used CBC encryption, every data block depended on an initialization vector anyone could predicted so someone listening nearby might grab sensitive information like session tokens. Even though modern browsers adopt workarounds

```
(gamebot@gamebot)-[~/cryptology/tls-project]
$ python3 tls_scanner.py

  TLS/SSL Vulnerability Scanner
  M542 Cryptology Project

[*] Scanning localhost:443 ...

[+] Results for localhost

[OK] SSLv2 Disabled
[OK] SSLv3 Disabled – POODLE safe
[HIGH] TLS 1.0 ENABLED – Vulnerable to BEAST!
[MEDIUM] TLS 1.1 ENABLED – Deprecated!
[OK] TLS 1.2 Supported
[INFO] TLS 1.3 Not supported
[OK] Not vulnerable to Heartbleed

⚠ 2 vulnerabilities found:
  → TLS 1.0 / BEAST
  → TLS 1.1 Deprecated
```

Figure 2: TLS Scanner Output - Vulnerability Report (localhost:443)

such as 1/n-1 record splitting, servers that support the outdated protocol remain exposed; disabling TLS 1.0 entirely removes this risk.

While TLS 1.1 avoids the specific flaw exploited in BEAST, its use was formally discontinued following IETF’s RFC 8996 in 2021. Its continued presence on a server often signals neglected maintenance, possibly tied to outdated configurations elsewhere. The scanner correctly flagged this issue as not critical but still notable.

When tested, the server showed no exposure to Heartbleed, thanks to the Nginx 1.14 image using a patched OpenSSL build. Keeping software current eliminates such vulnerabilities entirely.

Table 1: TLS/SSL Vulnerability Scan Results — Lab Server

Vulnerability	Protocol	Status	Notes
BEAST	TLS 1.0	Susceptible	CBC IV predictability
Deprecated Protocol	TLS 1.1	Flagged	Deprecated per RFC 8996
HEARTBLEED	OpenSSL	Not Detected	Patched OpenSSL build
SSLv3 / POODLE	SSLv3	Not Detected	Disabled in test config

7.4 Wireshark Packet Capture

Running the scan against the loopback interface, wireshark recorded live network data. What stands out is the TLS handshake process starting with ClientHello, followed by ServerHello, then certificates flying back and forth before ending in a finished signal. Notably, within that first ClientHello message hides key details: supported cipher suites along with available TLS versions offered up by the tools.

```
(gamebot@gamebot)-[~/cryptology/tls-project]
$ sudo wireshark
[sudo] password for gamebot:
** (wireshark:176651) 10:36:28.549918 [Capture MESSAGE] -- Capture Start ...
** (wireshark:176651) 10:36:28.570084 [Capture MESSAGE] -- Capture started
** (wireshark:176651) 10:36:28.570279 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackV
MwKL3.pcapng"
** (wireshark:176651) 10:47:01.956281 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:176651) 10:47:01.989216 [Capture MESSAGE] -- Capture stopped.
```

Figure 3: Wireshark - TLS Handshake Packet Capture

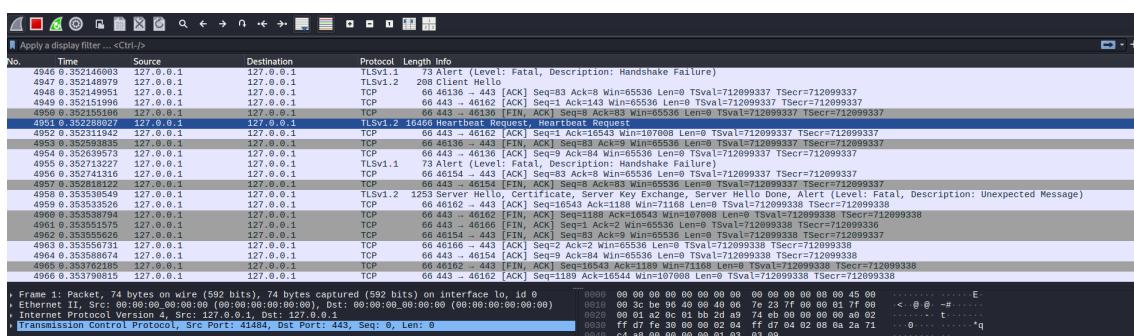


Figure 4: Wireshark — ClientHello Packet Detail (Cipher Suites Offered)

8. Critical Analysis

8.1 Why These Vulnerabilities Still Matter

Although the vulnerabilities were publicised between 2011 and 2014, TLS 1.0 and TLS 1.1 were still active on a significant percentage of internet facing servers as of 2020. Data from SSL Labs indicates that a notable proportion of servers continued to use deprecated versions of TLS well after their official deprecation. This inertia stems from compatibility requirements with legacy clients, operational momentum, and the invisibility of server configurations precisely the issue this scanner is designed to address.

Although the BEAST attack is partially mitigated by modern browsers, it can still be exploited against non-browser TLS clients such as API endpoints, IoT devices, and embedded systems that do not implement the 1/n-1 record splitting countermeasure. This necessitates the server-side blocking of TLS 1.0.

8.2 Cryptographic Design Lessons

A common theme among the attacks examined in this report is that they exploit the gap between the theoretical security of a cryptographic primitive and its practical implementation. Although CBC mode is mathematically sound, TLS 1.0's implementation violated the requirement for unpredictable IVs. Block ciphers require padding, but SSLv3 and TLS's error handling led to oracle vulnerabilities. In response to these lessons, TLS 1.3 has adopted Authenticated Encryption with Associated Data (AEAD) schemes, particularly AES-GCM, which by design eliminates IV predictability and padding oracle issues [4].

8.3 Recommendations

Stopping old protocols help reduce risk. Servers should run only modern encryption version. Rather than keeping outdated option, stick to strong standard, where possible, adopt the newest version of TLS. Stronger ciphers offer better protection. Tools building for checking settings can find weak spots early. Keeping software current blocks known exploits. Outdated code opens doors attackers may walk through. Fresh updaters close those paths. Security improves when system stay lean. Old features fade out so new ones take their place. Scanning often reveals hidden flaws. Protection grow behind consistent maintenance. New methods replace fragile ones without delay. Safety lives in small steady steps.

9. Conclusion

This project has shown that attacks on TLS/SSL protocols like BEAST, POODLE, LUCKY13, and HEARTBLEED are threat to online security. This report has shown why these flaws arise and how they are tied directly to specific design and implementation decisions within the TLS protocol stack.

A Python-built scanner identified two live issues on the test server: TLS 1.0 remaining active (susceptible to BEAST) and TLS 1.1 still running despite being officially deprecated. Network traffic captured through Wireshark confirmed the scanner's findings with empirical evidence, demonstrating real risk tied to weak configuration choices.

Real-world encryption security depends not just on strong mathematics, but on how well protocol versions are managed, how correctly cryptographic code is implemented, and how consistently systems are reviewed. TLS 1.3 stands out as a significant leap forward, cutting legacy complexity and using only modern authenticated encryption. Its widespread adoption is strongly recommended wherever technically feasible.

References

- [1] Al Fardan, N. and Paterson, K. (2013) 'Lucky Thirteen: Breaking the TLS and DTLS Record Protocols', *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 526–540.
- [2] Codenomicon (2014) *The Heartbleed Bug*. Available at: <http://heartbleed.com> (Accessed: 1 March 2026).
- [3] Moller, B., Duong, T. and Kotowicz, K. (2014) *This POODLE Bites: Exploiting the SSL 3.0 Fallback*. Available at: <https://www.openssl.org/~bodo/ssl-poodle.pdf> (Accessed: 1 March 2026).
- [4] Rescorla, E. (2018) *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc8446> (Accessed: 1 March 2026).
- [5] Rizzo, J. and Duong, T. (2011) 'Here Come The $\oplus$ Ninjas', *Unpublished manuscript*. Available at: <https://www.researchgate.net/publication/262167952> (Accessed: 1 March 2026).
- [6] Sarkar, S. and Fitzgerald, G. (2015) 'Attacks on SSL: A Comprehensive Study of BEAST, LUCKY-13, CRIME, BREACH, RC4 and Heartbleed', *Se-*

mantic Scholar. Available at: <https://www.semanticscholar.org/paper>
(Accessed: 1 March 2026).